

TRABALHO PRÁTICO DE MODELAGEM DE SOFTWARE - PARTE II

Sistema de Recomendação Inteligente de Grade Acadêmica

Gerência de Configuração com Git e GitHub
Diagrama de Implantação

Departamento Acadêmico de Ciência da Computação (DACC)
Professor: Wellington Moreira de Oliveira
Autores: João Augusto Ribeiro Borges Pedro Paulo Rezende Teixeira Rafael Teixeira Picardi
2026

Documento elaborado a partir dos requisitos, da apresentação do projeto e dos protótipos de interface disponibilizados pela equipe.

Resumo executivo

O sistema proposto recomenda uma grade semestral personalizada para o aluno, considerando histórico acadêmico, pré-requisitos, oferta de turmas, horários, carga horária máxima, disciplinas optativas e impacto na integralização curricular. Os protótipos enviados já evidenciam dois momentos da experiência: a seleção de objetivos e prioridades da recomendação e a visualização/ajuste da grade sugerida antes da confirmação da matrícula.

Este documento consolida os dois itens da Parte II solicitados: a gerência de configuração com Git e GitHub e o diagrama de implantação.

Premissa de arquitetura. Como os materiais enviados contêm protótipos HTML, mas ainda não definem a tecnologia final de backend, o diagrama utiliza uma arquitetura web orientada a API REST e descreve o runtime de forma independente de linguagem. O PostgreSQL aparece como SGBD de referência e pode ser substituído por outro SGBD relacional equivalente caso a equipe adote uma decisão diferente.

1. Gerência de Configuração com Git e GitHub

1.1 Objetivo e escopo

A gerência de configuração garante que a equipe trabalhe sobre versões identificadas, rastreáveis e recuperáveis dos artefatos do projeto. Para este sistema, o Git deve controlar não apenas o código-fonte, mas também os protótipos, diagramas, documentação de requisitos, scripts de banco de dados, testes e arquivos de infraestrutura. O GitHub será utilizado como repositório remoto, mecanismo de colaboração, controle de mudanças e registro das evidências do desenvolvimento.

1.2 Itens de configuração

ID	Item de configuração	Conteúdo controlado	Local sugerido
IC-01	Documentação de análise e projeto	Requisitos, casos de uso, diagramas, planejamento, atas e relatório final.	docs/
IC-02	Protótipos de interface	Arquivos Grade.html e GradeAberta.html, imagens e demais ativos de interface.	prototipos/ ou frontend/
IC-03	Frontend	Código que implementa telas, validações do cliente e consumo da API.	frontend/
IC-04	Backend e motor de recomendação	API, regras de elegibilidade, priorização, cálculo de grade e autenticação.	backend/
IC-05	Banco de dados	Modelo físico, migrações SQL, dados iniciais e scripts de carga.	database/
IC-06	Infraestrutura e configuração	Arquivos de ambiente, provisionamento, deploy e configurações de servidor.	infra/
IC-07	Testes e qualidade	Testes unitários, integração, massa de dados e relatórios de execução.	tests/

ID	Item de configuração	Conteúdo controlado	Local sugerido
IC-08	Automação do GitHub	Workflows de CI, modelos de issue e modelo de pull request.	.github/

1.3 Estrutura recomendada do repositório

A estrutura abaixo mantém os artefatos separados por responsabilidade, facilita revisão e evita que diagramas, dados e código fiquem dispersos no repositório.

```
recomendador-grade-academica/
|-- docs/           # requisitos, diagramas e relatório
|-- prototipos/     # Grade.html e GradeAberta.html
|-- frontend/       # aplicação cliente após implementação
|-- backend/        # API e motor de recomendação
|-- database/
|   |-- migrations/ # evolução versionada do banco
|   |-- seeds/      # dados iniciais e massa de testes
|-- infra/          # configurações de ambiente e deploy
|-- tests/          # testes automatizados
|-- .github/
|   |-- workflows/  # integração contínua
|   |-- ISSUE_TEMPLATE/ # solicitações e bugs
|   |-- PULL_REQUEST_TEMPLATE.md
|-- .gitignore
-- README.md
```

1.4 Linha de base, versionamento e releases

Uma linha de base é um conjunto de artefatos aprovado e identificável, que só pode ser alterado por meio do fluxo de mudança. A numeração seguirá o padrão SemVer (MAJOR.MINOR.PATCH), em que mudanças incompatíveis alteram MAJOR, funcionalidades compatíveis alteram MINOR e correções alteram PATCH.

Linha de base	Tag sugerida	Conteúdo e critério
B0 - Inicial	v0.0.1	Criação do repositório, README, .gitignore e configuração de equipes/permiteções.
B1 - Análise e protótipos	v0.1.0	Requisitos, apresentação, Grade.html e GradeAberta.html incluídos e identificados.
B2 - Projeto	v0.2.0	Diagrama de implantação, diagramas de projeto e decisões arquiteturais aprovadas.
B3 - Núcleo funcional	v0.3.0	Validação de pré-requisitos, elegibilidade e primeira recomendação executável.
RC - Candidata a entrega	v1.0.0-rc.1	Testes críticos executados, documentação revisada e pendências conhecidas registradas.
Release da entrega	v1.0.0	Versão estável marcada por tag e registrada em GitHub Releases.

1.5 Estratégia de branches

Branch	Finalidade	Regra de uso
main	Versão estável e liberável	Protegida. Recebe apenas pull requests de release/hotfix aprovados. Cada merge relevante recebe tag.
develop	Integração contínua da equipe	Concentra as funcionalidades aprovadas para a próxima versão de teste.
feature/USxx-descricao	Implementação de história ou requisito	Criada a partir de develop. Ex.: feature/US05-recomendacao-automatica.
fix/issue-n-descricao	Correção não urgente	Criada a partir de develop e vinculada a uma issue de bug.
release/x.y.z	Preparação de versão	Usada para estabilização, revisão de documentação e correções finais antes de main.
hotfix/x.y.z	Correção urgente em produção	Criada a partir de main; após correção, volta para main e develop.

1.6 Fluxo de mudança e revisão

Toda alteração deve possuir rastreabilidade desde a necessidade até a versão entregue. O fluxo adotado é:

1. Registrar a necessidade como GitHub Issue, vinculando-a a uma história de usuário, bug ou melhoria.
2. Classificar prioridade e incluir a issue no GitHub Project/Backlog do Sprint.
3. Criar uma branch curta a partir de develop, usando o padrão feature/, fix/ ou hotfix/.
4. Implementar a alteração, atualizar testes e, quando aplicável, documentação e diagrama.
5. Enviar commits pequenos e semanticamente claros para o repositório remoto.
6. Abrir Pull Request, vinculando a issue correspondente e descrevendo impactos, testes e evidências.
7. Executar revisão por pelo menos um integrante e checagens automatizadas antes do merge.
8. Realizar merge na branch de destino, fechar a issue e atualizar o quadro do projeto.

1.7 Padrão de commits

Os commits devem ser pequenos, autocontidos e escritos no imperativo. O prefixo informa a natureza da mudança e permite gerar um histórico compreensível.

```
feat(recomendacao): calcular prioridade de disciplinas gargalo
fix(horarios): impedir seleção de turmas com sobreposição
docs(implantacao): atualizar protocolos do diagrama
test(prerequisitos): cobrir disciplina com aproveitamento
chore(ci): incluir validação no pull request
refactor(api): separar consulta acadêmica do motor de recomendação
```

1.8 Uso do GitHub como controle de mudança

- GitHub Issues: cada história de usuário, correção ou melhoria recebe identificador, contexto, critérios de aceite, prioridade e responsável.

- GitHub Project: colunas Backlog, Pronto para desenvolvimento, Em andamento, Em revisão, Em teste e Concluído; cada cartão corresponde a uma issue.
- Pull Requests: devem conter descrição da alteração, issue relacionada, impacto em dados/API, testes executados e captura de tela quando houver alteração visual.
- Proteção de branches: bloquear push direto em main; exigir Pull Request, aprovação de revisão e conclusão das verificações de CI antes do merge.
- Tags e Releases: criar tag para cada linha de base e anexar notas de versão indicando funcionalidades, correções, riscos e instruções de rollback.
- Rastreabilidade: issue -> branch -> commits -> pull request -> tag/release. Este encadeamento possibilita auditoria do que mudou e por quê.

1.9 Qualidade, segurança e recuperação

Aspecto	Controle adotado
Segredos	Nunca versionar senhas, tokens, chaves de API ou arquivos .env. Versionar apenas .env.example sem valores sensíveis.
Acesso	Habilitar autenticação em dois fatores no GitHub e conceder permissão de escrita somente aos integrantes necessários.
Integração contínua	Em cada pull request: validação de código, testes, build e verificação de arquivos sensíveis. A ferramenta concreta será ajustada à tecnologia escolhida.
Banco de dados	Versionar apenas scripts/migrações; backups de dados reais devem ficar em armazenamento seguro, fora do repositório Git.
Rollback	Se uma versão falhar, restaurar a tag estável anterior, executar rollback de migração quando necessário e registrar uma issue de incidente.

1.10 Configuração inicial e rotina da equipe

```
# Após criar o repositório remoto no GitHub
git clone [URL_DO_REPOSITORIO]
cd recomendador-grade-academica
git checkout -b develop
git push -u origin develop

# Para iniciar uma história
git checkout develop && git pull origin develop
git checkout -b feature/US05-recomendacao-automatica
git add .
git commit -m "feat(recomendacao): gerar primeira grade válida"
git push -u origin feature/US05-recomendacao-automatica
```

Após o push, a equipe deve abrir um Pull Request para develop, relacionar a issue correspondente e aguardar revisão. O merge direto por linha de comando em main não é permitido.

2. Diagrama de Implantação

2.1 Objetivo do diagrama

O diagrama de implantação é um diagrama estrutural da UML que apresenta a topologia física de execução do software. Ele responde onde os artefatos serão instalados, quais ambientes

de execução são necessários, quais nós físicos participam da solução e como a comunicação ocorre entre eles. Neste projeto, o diagrama complementa os protótipos e os diagramas lógicos ao transformar a solução em uma visão de operação web.

2.2 Premissas utilizadas

- O sistema será disponibilizado como aplicação web, acessada por navegador em computador, notebook ou smartphone.
- Os arquivos Grade.html e GradeAberta.html representam o ponto de partida do frontend; na entrega final, deverão compor uma aplicação cliente hospedada no servidor web.
- A lógica de recomendação será exposta por uma API REST e executada no servidor de aplicação, separada da camada de interface.
- Os dados persistentes serão mantidos em SGBD relacional. O PostgreSQL foi adotado como referência para a implantação proposta.
- A comunicação com o sistema acadêmico institucional deve ocorrer por API oficial autenticada, sem acesso direto ao banco de dados institucional.
- O GitHub e o GitHub Actions são usados para armazenamento de código, qualidade e entrega; não compõem o caminho de execução do usuário final.

2.3 Diagrama de implantação proposto

A figura apresenta dispositivos, ambientes de execução, artefatos, componentes e protocolos de comunicação. As linhas contínuas representam comunicação em tempo de execução; a linha tracejada representa a entrega automatizada a partir do GitHub Actions.

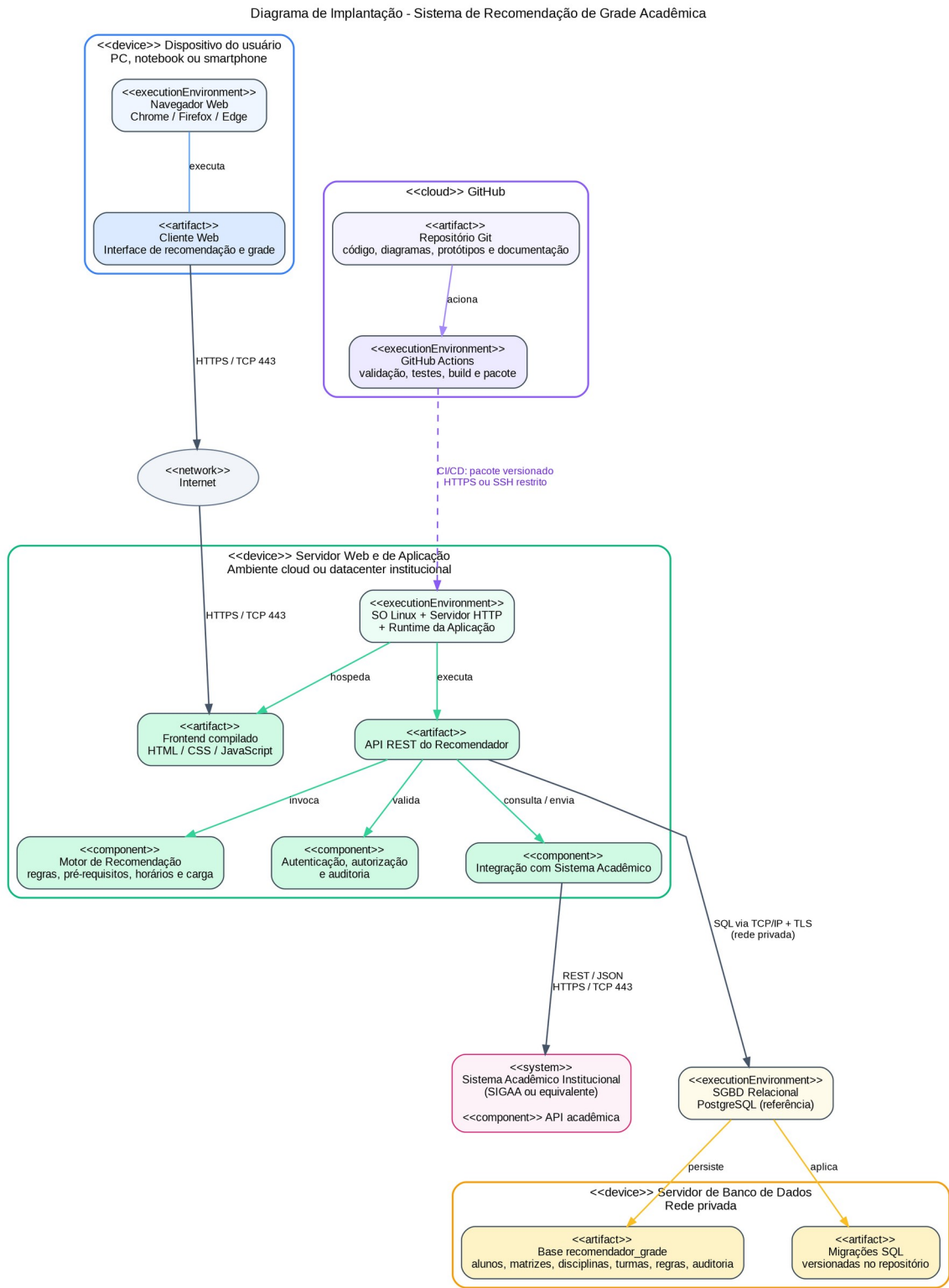


Figura 1 - Diagrama de implantação do Sistema de Recomendação de Grade Acadêmica. Fonte: elaborado pela equipe.

2.4 Elementos do diagrama

Nó/elemento	Tipo UML	Responsabilidade	Ambiente/artefatos
Dispositivo do usuário	<<device>>	Executa o navegador e apresenta o cliente web ao aluno ou administrador.	Navegador Web; Cliente Web.

Nó/elemento	Tipo UML	Responsabilidade	Ambiente/artefatos
Internet	<<network>>	Transporta as requisições públicas entre cliente e servidor.	HTTPS sobre TCP 443.
Servidor Web e Aplicação	<<device>>	Hospeda o frontend e executa a API, o motor de recomendação, autenticação e integração.	Servidor HTTP; runtime; frontend compilado; API REST.
Servidor de Banco de Dados	<<device>>	Mantém os dados do recomendador em rede privada, sem exposição pública direta.	SGBD; base recomendador_grade; migrações SQL.
Sistema Acadêmico	<<system>>	Fonte oficial de histórico, matriz, turmas, horários e destino da confirmação quando a integração for autorizada.	API acadêmica.
GitHub	<<cloud>>	Centraliza versão do código, protótipos, documentos e automação de build/teste/deploy.	Repositório Git; GitHub Actions.

2.5 Conexões e protocolos

Origem -> destino	Protocolo	Finalidade e cuidado
Navegador -> Servidor Web	HTTPS / TCP 443	Entrega da interface e chamadas autenticadas à API. Todo tráfego externo deve usar TLS.
API REST -> Banco de Dados	SQL via TCP/IP + TLS em rede privada	Persistência de regras, dados replicados/autorizados, recomendações e auditoria. A porta do banco não deve ficar pública.
Integração -> Sistema Acadêmico	REST / JSON via HTTPS	Consulta de histórico, matriz, pré-requisitos, turmas e horários; envio de confirmação apenas quando houver contrato/API oficial.
GitHub Actions -> Servidor	HTTPS ou SSH restrito	Publicação de pacote versionado após verificações de qualidade; credenciais ficam em GitHub Secrets.
Equipe -> GitHub	HTTPS ou SSH autenticado	Clone, push, pull request, revisão, registro de issues e criação de releases.

2.6 Fluxo operacional da recomendação

1. O usuário acessa o cliente web e informa meta de formação, carga horária máxima e prioridades, como evidenciado nos protótipos.
2. O navegador envia a solicitação autenticada para a API REST usando HTTPS.
3. A API consulta os dados necessários no banco local e, quando permitido, solicita ao sistema acadêmico informações atualizadas de histórico, matriz, oferta e horários.
4. O motor de recomendação filtra disciplinas já aprovadas, valida pré-requisitos, remove conflitos de horário, respeita carga máxima e prioriza componentes que aceleram a integralização.
5. A grade sugerida, as justificativas e os alertas de conflito são devolvidos ao cliente web para ajuste manual ou substituição de turma.

6. A confirmação só é processada após validação final. Se houver integração autorizada, a API encaminha a solicitação ao sistema acadêmico; caso contrário, registra a seleção para conferência manual.

2.7 Infraestrutura mínima de referência

Elemento	Requisito mínimo / cuidado
Cliente	Navegador atualizado e conexão estável com Internet. Não é necessário instalar aplicação local.
Servidor Web e Aplicação	Ambiente Linux, 2 vCPU, 4 GB RAM e 20 GB de armazenamento para protótipo acadêmico. A capacidade deve ser revisada após testes de carga.
Banco de Dados	2 vCPU, 4 GB RAM, armazenamento persistente criptografado e backup automático. Acesso permitido apenas ao servidor de aplicação e à administração autorizada.
Rede e segurança	HTTPS obrigatório; firewall liberando 443 para usuários; banco isolado em rede privada; acesso administrativo via VPN ou SSH com chave e restrição de IP.
Observabilidade	Logs de aplicação, health check, monitoramento de disponibilidade, contagem de erros, auditoria de alterações de regras e alertas de falha na integração acadêmica.

2.8 Plano resumido de implantação e suporte

Fase	Atividades principais	Critério de saída
1. Preparação	Criar ambientes de desenvolvimento, teste e produção; configurar domínio, HTTPS, variáveis de ambiente, GitHub Secrets e backups.	Ambiente acessível apenas à equipe e checklist técnico aprovado.
2. Carga inicial	Cadastrar/replicar matriz, disciplinas, pré-requisitos, turmas e regras de prioridade; executar migrações versionadas.	Base consistente e validada com cenários de teste.
3. Publicação	Executar pipeline de build/teste; publicar artefato identificado por tag; aplicar migrações de forma controlada.	Aplicação disponível e health check aprovado.
4. Homologação	Testar consulta de histórico, geração de grade, conflitos, carga horária, prioridades e integração acadêmica.	Casos críticos aprovados e defeitos bloqueadores resolvidos.
5. Adoção e suporte	Disponibilizar guia de uso, canal de dúvidas e rotina de monitoramento de uso, erros e solicitações de mudança.	Usuários orientados e indicadores acompanhados.
6. Evolução	Registrar bugs e melhorias como issues; priorizar no backlog e liberar novas versões por tags/releases.	Histórico de mudança rastreável e rollback possível.

2.9 Controles de segurança e continuidade

- Autenticação e autorização devem ser aplicadas no backend; o frontend não pode decidir sozinho quais dados acadêmicos o usuário pode consultar.
- Informações acadêmicas devem trafegar por HTTPS, ter acesso baseado em perfil e gerar registros de auditoria para consultas e confirmações relevantes.
- O banco de dados deve receber backup periódico testado por restauração. Git não substitui backup de dados operacionais.

- As migrações SQL devem ser aplicadas de modo versionado e reversível, evitando alterações manuais não documentadas no ambiente de produção.
- A implantação deve manter uma versão estável marcada por tag para rollback rápido em caso de erro após uma atualização.
- Logs devem apoiar diagnóstico, mas evitar exposição desnecessária de dados pessoais e históricos completos do aluno.

3. Pendências a validar antes da entrega final

A proposta é completa para a fase de projeto, mas os pontos abaixo precisam ser confirmados pela equipe e pela instituição antes de uma implantação real:

- URL e organização do repositório GitHub que serão utilizados pela equipe.
- Tecnologia final do backend e do frontend após a transição dos protótipos HTML.
- Hospedagem adotada: infraestrutura institucional, cloud, servidor próprio ou ambiente acadêmico.
- Contrato de integração, endpoints, autenticação e limites da API do sistema acadêmico.
- Política institucional para dados acadêmicos, perfis de acesso, retenção de logs e procedimento de homologação.

Referências

OLIVEIRA, Wellington Moreira de. Trabalho Prático de Modelagem de Software. Departamento Acadêmico de Ciência da Computação. Material de aula, 2026.

OLIVEIRA, Wellington Moreira de. Projeto de Implantação de Software. Departamento Acadêmico de Ciência da Computação. Material de aula, 2026.

EQUIPE DO PROJETO. Apresentação do Sistema de Recomendação de Grade Acadêmica. Material interno, 2026.

EQUIPE DO PROJETO. Protótipos de interface Grade.html e GradeAberta.html. Material interno, 2026.

Fim do documento.